ReDeEP: Detecting Hallucination in Retrieval-Augmented Generation via Mechanistic Interpretability

A Reproduction and Robustness Study

Gourilakshmi S
23BCS21
gourilakshmis78@gmail.com

Adithya K
23BCS51
aditkrishna95@gmail.com

***Abstract*—Retrieval-augmented generation (RAG) is meant to ground a language model in retrieved evidence, yet models still hallucinate even when the evidence is correct. ReDeEP (Sun et al., ICLR 2025) detects such hallucinations by reading a model's internals: it measures how much *copying heads* use the retrieved context and how much *knowledge FFNs* inject parametric memory. We reproduce ReDeEP on the RAGTruth benchmark with a single 6 GB laptop GPU. On the paper's own backbone, Llama-2-7B-chat, our detector reaches an AUC of 0.739 against the published 0.7325, and the paper's two mechanistic findings replicate: all 1024 heads show lower context usage in hallucinated responses, and later-layer FFNs inject more parametric knowledge. Across four LLaMA-family models from 1.1B to 7B parameters, ReDeEP outperforms perplexity and entropy baselines by 0.06 to 0.11 AUC, although per-task AUCs are markedly lower than the pooled figure. We then stress-test the method under corrupted retrieval. An irrelevant passage raises the hallucination score sharply, but corrupting the passage's numbers, so that the response contradicts it, leaves the score unchanged or lowers it: ReDeEP measures whether a response is *about* its context, not whether it *agrees* with it. Finally, prompted black-box judges from the Gemini and GPT families (AUC 0.827 and 0.790) outperform ReDeEP (0.721) on the same held-out responses, while a small 3B LLaMA judge performs at chance (0.499).**

***Index Terms*—hallucination detection, retrieval-augmented generation, mechanistic interpretability, large language models, robustness, reproducibility**

I. Introduction

Large language models (LLMs) answer fluently but are prone to *hallucination*: generating content that is not supported by any evidence. Retrieval-augmented generation (RAG) is the standard remedy. A retriever fetches passages relevant to the user's query, and the model is instructed to answer from them. RAG is therefore widely deployed where trust matters, such as legal research, medical question answering and enterprise search.

RAG does not remove the problem. Even when the retrieved passage is accurate and contains the answer, models still produce claims that contradict or go beyond it. A model may be given a passage stating that a lunar mission returned 1,935.3 g of samples and still answer with a different number recalled from its training data. If a model can hallucinate when the correct answer is in front of it, adding retrieval is not a sufficient fix. We also need detectors that tell when a response has drifted away from its context.

ReDeEP [1] approaches this through *mechanistic interpretability*: rather than judging the output text, it inspects how the model produced it. Its central hypothesis is that two internal components compete. *Copying heads*, attention heads that move information from context tokens into the residual stream, carry the external knowledge. *Knowledge FFNs*, feed-forward layers in the later part of the network, inject the model's stored, parametric knowledge. A RAG hallucination occurs when the model under-uses the first and over-relies on the second. ReDeEP turns both effects into scores and combines them into a hallucination detector.

Our review of the method, prepared for this project, identified four gaps: it is validated only on LLaMA-family models, its head and layer selection is tuned by hand, its mitigation (AARF) gives only modest gains, and it *assumes the retrieved context is accurate*, never testing noisy or wrong retrieval, a common failure in deployed systems. This project reproduces the method and then targets the last gap directly.

Contributions.

1) A reproduction of ReDeEP's token-level detector on RAGTruth [2] on consumer hardware (RTX 4050, 6 GB VRAM). Memory optimisations make 7B models feasible, and the result matches the published AUC.
2) A replication of the paper's two mechanistic findings, and a transfer study across four LLaMA-family backbones [3] from 1.1B to 7B parameters, with ROC, per-task and ablation analyses.
3) A robustness extension that holds each response fixed and corrupts the retrieved passage, showing that ReDeEP detects off-topic context but not contradicted facts.
4) A comparison against prompted black-box judges from the GPT, Gemini and LLaMA families on the same held-out responses.

II. Background and Related Work

A. RAG Hallucination versus Knowledge Conflict

ReDeEP distinguishes two failure modes. A *knowledge conflict* arises when the retrieved context itself is wrong or contradicts what the model knows. A *RAG hallucination* arises when the context is accurate and relevant, yet the response contradicts it or adds unsupported content. The paper targets only the second. Its datasets are chosen so that the context can be trusted, and the response is the only thing that can be wrong. Our robustness extension (Section VIII) deliberately crosses this boundary to see how the method behaves just outside its stated scope.

B. Families of Hallucination Detectors

Sun et al. group prior detectors into three families by which signal they read and what confounds it:

- *Parametric confounded by external (PCE)* methods read hidden states to estimate what the model knows, for example EigenScore, SEP, SAPLMA and ITI. The external context mixes into these states and confounds the reading.
- *External confounded by parametric (ECP)* methods check the response against the context with prompts or retrieval-style checks, for example RAGAS, ChainPoll and RefChecker. The checking model's own knowledge leaks into the judgement.
- *Mixed parametric and external (MPE)* methods use uncertainty from sampling or token probabilities, for example perplexity, energy and LN-Entropy [4]. They blend both knowledge sources without separating them.

None of these *decouples* context usage from parametric-knowledge usage. ReDeEP's claim is that decoupling them gives a cleaner signal. In our experiments, perplexity and LN-Entropy represent the MPE family, and prompted LLM judges represent the ECP family.

C. Mechanistic Interpretability Tools

Transformer language models maintain a *residual stream*: each layer reads it, and its attention and FFN blocks add their outputs back into it. Two tools let ReDeEP read this stream. First, each attention head's *OV circuit* ($W_V W_O$) describes what information the head writes when it attends to a token. A head whose OV circuit maps a token's embedding back towards the same token's unembedding is a *copying head*. Second, the *logit lens* [6] applies the model's final norm and unembedding W_U to an intermediate residual state, giving the next-token distribution the model "would predict" at that depth. Prior work shows that FFN layers act as key-value memories that store factual knowledge [5], which motivates measuring what each FFN adds.

III. The ReDeEP Method

Fig. 1 summarises the detector. Given a prompt (instruction plus retrieved passage) and a response, ReDeEP runs one forward pass over their concatenation. Let R be the set of response token positions and C the set of context positions.

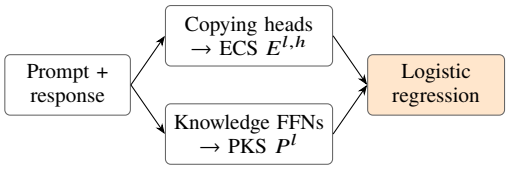


Fig. 1. ReDeEP pipeline: one forward pass yields an external-context score per head and a parametric-knowledge score per FFN layer; a logistic regression combines them into a hallucination score.

A. External Context Score (ECS)

For head h in layer l and response token n, let $\mathcal{T}_n \subset C$ be the 10% of context tokens that the head attends to most. With x_i^L the final-layer hidden state of token i,

$$E^{l,h} = \frac{1}{|R|} \sum_{n \in R} \cos\Big(\tfrac{1}{|\mathcal{T}_n|} \textstyle\sum_{i \in \mathcal{T}_n} x_i^L,\ x_n^L \Big). \tag{1}$$

ECS asks whether the content a head read is reflected in what the model wrote, not merely whether it attended to the right place. The paper finds that about 77.5% of hallucinations occur even though the heads attend to the correct context, so attention location alone is not enough.

B. Parametric Knowledge Score (PKS)

For layer l, the residual state just before the FFN block ($x^{\text{mid},l}$) and just after the whole layer (x^l) are decoded with the logit lens. With $p_n = \sigma(W_U \,\text{norm}(x_n))$ the resulting next-token distribution,

$$P^l = \frac{1}{|R|} \sum_{n \in R} \text{JSD}\big(p_n^{\text{mid},l},\ p_n^l\big), \tag{2}$$

where σ is the softmax and JSD the Jensen–Shannon divergence. A large divergence means the FFN changed what the model wanted to say, so it injected its own knowledge.

C. Selecting Copying Heads and Knowledge FFNs

Copying heads are found from weights alone. For each head we form the matrix $M = W_E\, W_V^h W_O^h\, W_U^\top$ restricted to a fixed random sample of 500 vocabulary tokens. A copying head has a large-magnitude trace, meaning tokens map strongly onto themselves, and few Gershgorin-disc outliers, meaning a well-conditioned diagonal. Heads are ranked by the sum of both ranks, and the top $|\mathcal{A}| = 8$ form the set $\mathcal{A}$. Knowledge FFNs are chosen with labels. On each training fold, the $|\mathcal{F}| = 5$ layers whose PKS correlates most positively with the hallucination label form the set $\mathcal{F}$.

D. Detector

ReDeEP models the hallucination score as $H = \alpha\, \bar{P}_{\mathcal{F}} - \beta\, \bar{E}_{\mathcal{A}}$, with $\alpha, \beta > 0$, where the bars denote means over the selected layers and heads. We fit this as a logistic regression on the two standardised features $\bar{P}_{\mathcal{F}}$ and $-\bar{E}_{\mathcal{A}}$. Its probability output is the score, and the decision threshold is the best-F1 point on the training fold. Algorithm 1 gives the scoring procedure.

Algorithm 1 ReDeEP scoring of one response

Require: prompt c, response r, heads $\mathcal{A}$, layers $\mathcal{F}$, fitted regression g

run one forward pass on $[c; r]$; hooks record per layer the top-k attended context indices and $x^{\text{mid},l}$
for each head $(l, h) \in \mathcal{A}$ **do**
    $E^{l,h} \leftarrow$ mean over $n \in R$ of $\cos(\text{mean}_{i \in \mathcal{T}_n} x_i^L,\ x_n^L)$
end for
for each layer $l \in \mathcal{F}$ **do**
    $P^l \leftarrow$ mean over $n \in R$ of $\text{JSD}(p_n^{\text{mid},l}, p_n^l)$
end for
return $g\big(\text{mean}_{\mathcal{F}} P,\ -\text{mean}_{\mathcal{A}} E\big)$

E. AARF Mitigation

The paper also proposes AARF ("Add Attention, Reduce FFN"). At decoding time it scales copying-head outputs by $\alpha_2 > 1$ and knowledge-FFN outputs by $\beta_2 < 1$, steering generation towards the context without fine-tuning. Our code base implements AARF through forward hooks. Its evaluation needs an LLM judge of generated text and was outside the scope and compute budget of this study, which focuses on detection.

IV. Dataset

A. RAGTruth

RAGTruth [2] is the first large word-level hallucination corpus built specifically for RAG. It contains 17,790 responses to 2,965 source prompts. Each response was generated by one of six LLMs: GPT-3.5-turbo, GPT-4, Llama-2-7B/13B/70B-chat and Mistral-7B-Instruct. Human annotators marked every hallucinated span with its character offsets and type. It covers three task types: question answering over MS MARCO passages, summarisation of news articles, and data-to-text generation from structured Yelp business records.

The corpus has two files. `source_info.jsonl` holds each source's identifier, task type and full prompt including the retrieved context. `response.jsonl` holds each response with its generator model, train/test split, a quality flag for refusals or truncation, and the list of hallucination spans. Following the paper, a response is labelled hallucinated if it contains at least one span. We keep responses flagged as good quality.

Evaluation subset. The paper evaluates each backbone on responses that backbone generated. We use the Llama-2-7B-chat setting: all good-quality *test-split* responses generated by llama-2-7b-chat, capped at 6,000 characters of prompt plus response so that every example fits in 1,024 tokens. This yields 415 responses, of which 212 (51%) are hallucinated. Table I breaks them down by task.

B. Dolly (AC)

The paper also evaluates on Dolly (AC), a subset of Databricks' dolly-15k instruction data. It is filtered to tasks with an accurate context passage (closed QA, summarisation and information extraction), regenerated by the backbone, and hand-labelled by annotators. Dolly-15k itself has no hallucination labels, and the annotated Dolly (AC) responses are not distributed with the public data we used. We therefore restrict our study to RAGTruth.

TABLE I
RAGTruth by task: full corpus and our evaluation subset

		Full corpus		Our subset	
Task	Source	Resp.	Halluc.	Resp.	Halluc.
Question answering	MS MARCO	5,934	29%	149	35%
Summarization	CNN/DM, news	5,658	30%	121	35%
Data-to-text	Yelp	6,198	69%	145	81%

TABLE II
Backbones

Backbone	Params	Layers×Heads	Vocab	Precision
TinyLlama-1.1B-Chat	1.1B	22×32	32k	fp16
Llama-3.2-1B-Instruct	1.2B	16×32	128k	fp16
Llama-3.2-3B-Instruct	3.2B	28×24	128k	4-bit NF4
Llama-2-7B-chat	6.7B	32×32	32k	4-bit NF4

V. Experimental Setup

A. Backbones

Table II lists the four LLaMA-architecture backbones. All share the pre-norm decoder with RMSNorm, rotary embeddings and SwiGLU FFNs, so the same code applies to each. Llama-3.2 uses grouped-query attention, which our copying-head analysis handles by pairing each query head with its shared key-value head. Weights are loaded from public re-uploads of the official releases, so no gated licence is required.

B. Hardware and Software

All experiments ran on a Lenovo LOQ laptop with an NVIDIA RTX 4050 Laptop GPU (6 GB VRAM) and 16 GB RAM, under Windows 11. We used PyTorch 2.11 with CUDA 12.8, Hugging Face transformers 5.17 and bitsandbytes 0.50. The 3B and 7B models were loaded in 4-bit NF4 with fp16 computation; the smaller models ran in fp16.

C. Engineering for a 6 GB GPU

The reference pipeline asks the model to return every full attention matrix. For Llama-2-7B at 1,024 tokens that is 32 layers × 32 heads × 1024^2 values, which does not fit beside the weights. We made three changes:

1) Forward hooks on each attention module keep only the top-k context indices per head and response token, then discard the attention map.
2) The mean of attended hidden states is computed as a sparse averaging-matrix product instead of gathering a $[H, |R|, k, d]$ tensor.
3) Perplexity and LN-Entropy reuse the logits of the same forward pass instead of a second pass.

On a RAGTruth example, the rewritten ECS matched the original implementation to within 6×10^{-8}, so results remain comparable. Copying-head weights are dequantised from 4-bit on the fly. Prompts longer than the budget are truncated from the left, so the whole response is always scored.

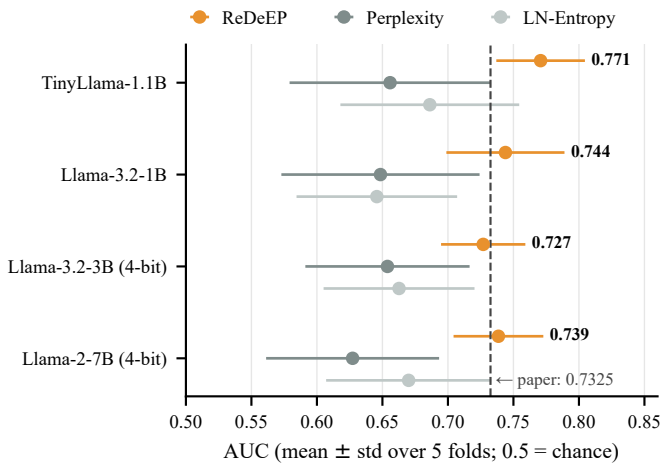


Fig. 2. Detection AUC per backbone (mean ± std over 5 folds). ReDeEP (orange) beats both baselines on every model; the dashed line marks the paper's Llama-2-7B result.

D. Evaluation Protocol

We report AUC (threshold-free ranking quality), the Pearson correlation (PCC) between score and label, and recall and F1 at the training-fold threshold. Our main protocol is stratified 5-fold cross-validation over the 415 responses, reported as mean ± standard deviation over folds. Copying heads are fixed from weights, and knowledge FFNs and the regression are re-fitted inside each training fold, so no test label influences selection. For comparisons with the judges we also use a single fixed 70/30 split, whose test part has 124 responses.

E. Baselines

Perplexity is the exponentiated mean negative log-likelihood of the response tokens given the prompt. **LN-Entropy** is the mean per-token entropy of the predictive distribution over the response [4]. Both come from the same forward pass and backbone as ReDeEP. Higher values are taken to indicate hallucination.

F. Black-Box Judges

Each judge receives the RAG input and the response and replies in JSON with the probability (0–100) that the response contains a hallucination (prompt in Appendix B). We used Gemini 3.5 Flash-Lite through the Google AI Studio free tier. For the GPT family we used OpenAI's open-weights gpt-oss-120b, served free by Groq. We also ran Llama-3.2-3B-Instruct locally in 4-bit. The stated probability is the score.

VI. Results

A. Hallucination Detection

ReDeEP reproduces on the paper's backbone: Llama-2-7B reaches an AUC of 0.739 (±0.034) against the published 0.7325, and a PCC of 0.427 against 0.3979 (Table III, Fig. 2). It beats both baselines on every backbone. The margin over the better baseline ranges from 0.06 (Llama-3.2-3B against LN-Entropy) to 0.10 (Llama-3.2-1B against perplexity). Detection

TABLE III
Detection results, 5-fold CV mean (n = 415)

	ReDeEP			PPL	LN-Ent
Backbone	AUC	PCC	F1	AUC	AUC
TinyLlama-1.1B	**0.771**	0.475	0.718	0.656	0.686
Llama-3.2-1B	**0.744**	0.426	0.692	0.649	0.646
Llama-3.2-3B (4-bit)	**0.727**	0.399	0.695	0.654	0.663
Llama-2-7B (4-bit)	**0.739**	0.427	0.687	0.627	0.670
Paper (Llama-2-7B)	0.7325	0.3979	–	–	–

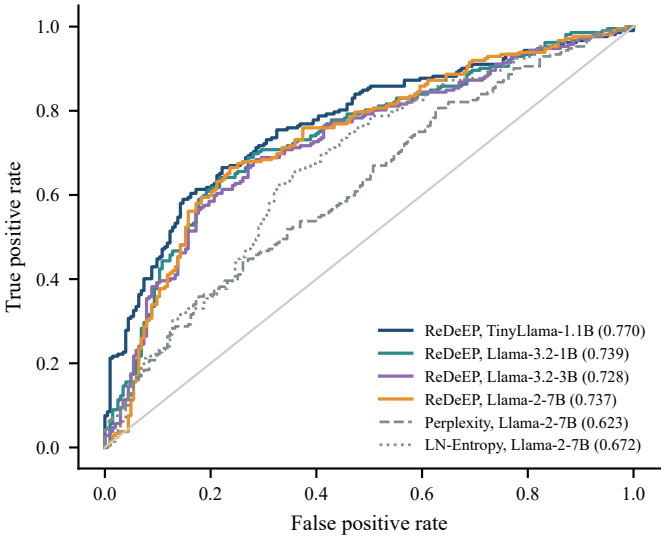


Fig. 3. ROC curves from out-of-fold scores. All four ReDeEP curves lie above the perplexity and LN-Entropy curves of Llama-2-7B across most of the range.

quality is roughly flat in model size: the 1.1B TinyLlama scores highest, and the 3B and 7B models lie within one standard deviation of each other. ReDeEP is also more stable across folds (std 0.03–0.05) than the baselines (0.06–0.08).

Fig. 3 shows ROC curves built from out-of-fold predictions. ReDeEP dominates the baselines over most of the false-positive range rather than only at one operating point. Fig. 4 shows that its scores separate the two classes but overlap heavily in the middle, which matches an AUC in the mid-0.7s.

B. Replicating the Mechanistic Findings

The paper reports two core observations on Llama-2-7B: copying heads use less context in hallucinated responses (1006 of 1024 heads), and later-layer FFNs inject more parametric knowledge. Both replicate (Table IV). On Llama-2-7B, all 1024 of 1024 heads have higher mean ECS on truthful responses (Fig. 5). PKS is higher on hallucinated responses in 16 of the 16 later layers, but only 6 of the 16 earlier ones (Fig. 6). TinyLlama and Llama-3.2-3B show the same pattern clearly; Llama-3.2-1B shows it more weakly (4 of 8 later layers against 1 of 8 earlier ones).

One caution applies to the ECS result. Because ECS compares final-layer hidden states for every head, the per-

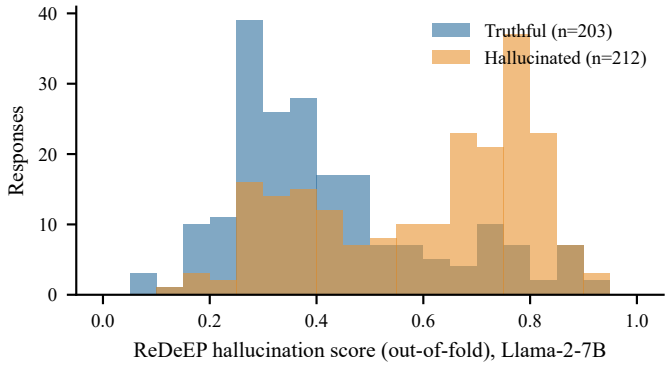


Fig. 4. Distribution of out-of-fold ReDeEP scores on Llama-2-7B. Hallucinated responses shift right, with substantial overlap in the middle.

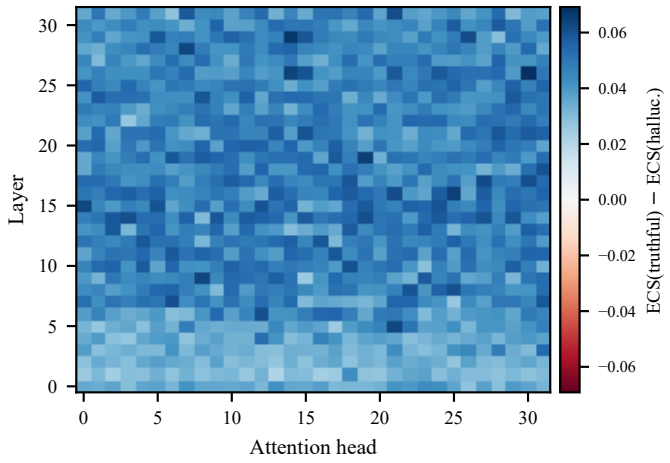


Fig. 5. Mean ECS of truthful minus hallucinated responses for every head of Llama-2-7B. Every cell is positive: context usage is lower in hallucinated responses across all heads.

head scores of a response are strongly correlated. The near-unanimous head count therefore reflects a shift at the level of the whole response more than 1024 independent effects.

C. Performance by Task

Pooled AUC hides large differences between tasks (Table V, Fig. 7). Within a single task, ReDeEP's AUC falls to between 0.40 and 0.66. On Llama-2-7B it is 0.623 on QA, 0.561 on summarisation and 0.396 on data-to-text. ReDeEP beats the baselines on QA for every backbone, but the baselines win on summarisation for three of the four models, and on data-to-text ReDeEP is near or below chance for the three Llama models.

Part of the pooled score therefore comes from separating *tasks*, not responses. The hallucination rate differs strongly by task in our subset: 35% for QA, 35% for summarisation and 81% for data-to-text. Internal signals also differ systematically by task. Data-to-text prompts are structured JSON, whose hidden states plausibly look unlike prose, so a detector may partly identify the task and inherit its base rate. The paper reports pooled numbers too, so the comparison with it is

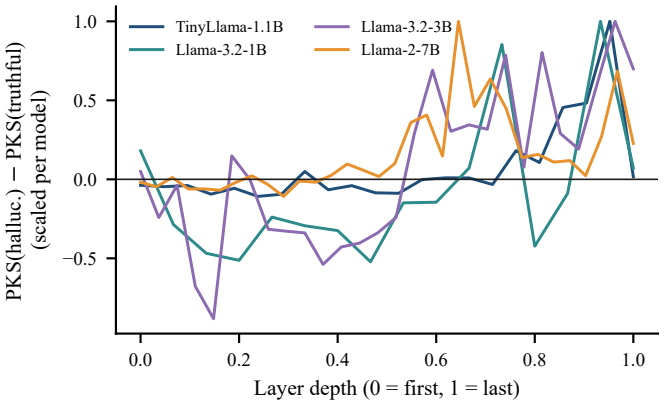


Fig. 6. Mean PKS of hallucinated minus truthful responses by relative layer depth. The difference is concentrated in the later layers, where knowledge FFNs sit.

TABLE IV
REPLICATION OF THE PAPER'S TWO MECHANISTIC FINDINGS

		Heads: ECS truth >	Layers: PKS halluc. > truth	
Backbone	$L \times H$	halluc.	Early	Late
TinyLlama-1.1B	22×32	704/704	1/11	8/11
Llama-3.2-1B	16×32	512/512	1/8	4/8
Llama-3.2-3B	28×24	672/672	2/14	13/14
Llama-2-7B	32×32	1024/1024	6/16	16/16
Paper (Llama-2-7B)	32×32	1006/1024	higher in later layers	

fair, but within-task evaluation is the more demanding test for deployment.

D. Ablations

Both signals matter. Table VI fits the detector on each feature alone. On Llama-2-7B, ECS alone reaches 0.711 and PKS alone 0.708, while combining them gives 0.737. The combination is the best on every backbone, which supports ReDeEP's central claim that context usage and parametric injection are complementary. Which signal is stronger depends on the model: PKS dominates on TinyLlama, and ECS on the Llama-3.2 models.

Sensitivity to $|\mathcal{A}|$ and $|\mathcal{F}|$. One of the gaps we identified was that head and layer selection is tuned by hand. Fig. 8 sweeps 1 to 32 copying heads and 1 to 10 FFN layers on Llama-2-7B. AUC stays between 0.736 and 0.751 across the whole grid, against 0.737 for our default. The detector is therefore insensitive to these hyperparameters in our setting, and the hand-tuning concern matters less than expected. This is consistent with the correlated per-head ECS noted above.

E. Computational Cost

Table VII reports wall-clock costs. Copying heads are computed once per model from weights in a few seconds. Scoring takes a median of about half a second per response for TinyLlama and 4.5 s for Llama-2-7B in 4-bit. The complete Llama-2-7B run over 415 responses took 38 minutes on a

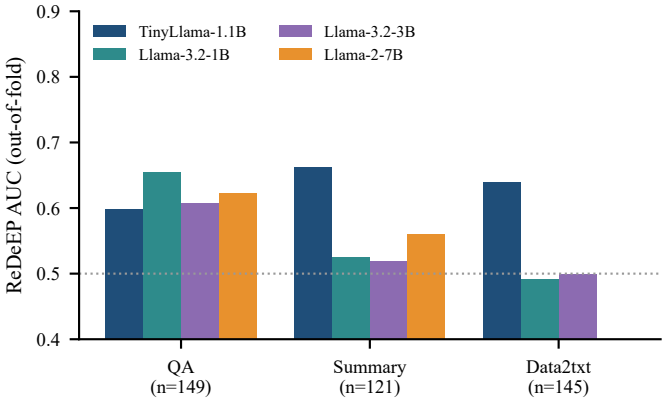


Fig. 7. Out-of-fold ReDeEP AUC within each task type. Dotted line: chance.

TABLE V
AUC WITHIN EACH TASK: REDEEP / BEST BASELINE (BOLD: REDEEP WINS)

Backbone	QA	Summary	Data-to-text
TinyLlama-1.1B	**0.598** / 0.582	**0.662** / 0.659	**0.639** / 0.458
Llama-3.2-1B	**0.655** / 0.563	0.525 / 0.668	**0.492** / 0.419
Llama-3.2-3B	**0.608** / 0.552	0.519 / 0.680	**0.499** / 0.465
Llama-2-7B	**0.623** / 0.539	0.561 / 0.652	0.396 / 0.505

laptop. ReDeEP needs no sampling, no second model and no network access.

VII. Comparison with Black-Box Judges

Large judges win on accuracy. Gemini 3.5 Flash-Lite reaches 0.827 AUC and gpt-oss-120b 0.790, against 0.721 for ReDeEP on the same 124 responses (Table VIII, Fig. 9). On all 415 responses Gemini scores 0.810, consistent with the held-out result. The judges read the passage and the response as text, so they can directly spot a number or name that does not match, which is exactly the weakness Section VIII exposes in ReDeEP. Both judges have very high recall (0.91–0.93): they flag most hallucinations, at the cost of more false alarms.

Judge scores are coarse. Although asked for a probability, Gemini answered exactly 0 or 100 for 99.5% of responses, which turns its output into a near-binary verdict. gpt-oss-120b used 17 distinct values. A finer-grained score would likely raise both judges' AUC further.

Small judges fail. Llama-3.2-3B answered 0 or 100 for 99.8% of responses and almost always said "no hallucination", catching 7% of hallucinations (AUC 0.499). Reading the internals of a model of the same size (ReDeEP on Llama-3.2-3B, AUC 0.727) works far better than asking it.

Trade-off. The judges rely on large models served behind an API, with per-call cost, rate limits and data leaving the machine. Our free-tier access was limited to 20 requests per day for the newest Gemini Flash model, which forced the use of Flash-Lite. ReDeEP runs locally on a laptop with a 7B open model. The practical choice depends on whether accuracy or cost and privacy dominate.

TABLE VI
FEATURE ABLATION: OUT-OF-FOLD AUC

Backbone	ECS only	PKS only	Both
TinyLlama-1.1B	0.727	0.749	**0.770**
Llama-3.2-1B	0.717	0.660	**0.739**
Llama-3.2-3B	0.717	0.635	**0.728**
Llama-2-7B	0.711	0.708	**0.737**

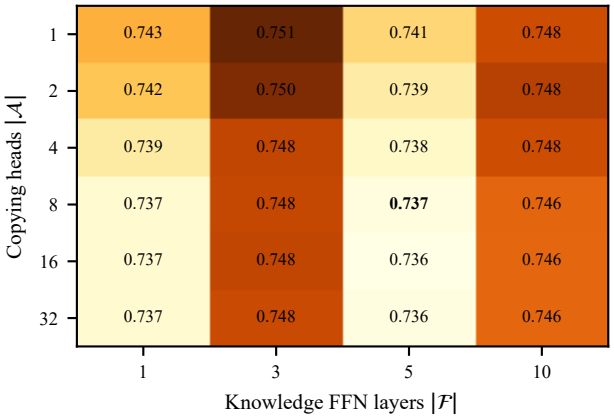


Fig. 8. Out-of-fold AUC on Llama-2-7B as the number of copying heads and knowledge-FFN layers varies. Bold: the default (8 heads, 5 layers).

VIII. Extension: Robustness Under Corrupted Retrieval

ReDeEP assumes accurate context. We ask what its signals measure when that assumption is broken on purpose. Two outcomes are possible. If ECS and PKS track *faithfulness to the given context*, a response should look hallucinated whenever its context stops supporting it. If they track something weaker, such as topical similarity, some kinds of contradiction will go unnoticed.

A. Design

We hold each response fixed and change only the retrieved passage, under three conditions:

1) *Accurate:* the real RAGTruth passage (control).
2) *Irrelevant:* a passage from a different source of the same task type. Nothing in the response is supported any more.
3) *Numbers corrupted:* every number in the real passage is multiplied by a random factor in $[1.3, 3.0]$, with a fixed seed per example. Numbers the response copied are now contradicted by the passage, while the topic and wording are unchanged.

We draw 150 responses (75 hallucinated), refit the detector on the remaining 265, and score each response under all three passages with TinyLlama-1.1B and Llama-2-7B. A faithful detector should raise its score under both corrupted conditions. We test the per-response score change with a paired Wilcoxon signed-rank test.

B. Results

Irrelevant passages work as intended. The score jumps for nearly every response, by +0.317 for TinyLlama and +0.250

TABLE VII
COMPUTE COST ON AN RTX 4050 (415 RESPONSES)

Backbone	Copying heads (s)	Per response (s)	Total (min)
TinyLlama-1.1B	3.7	0.5	3.7
Llama-3.2-1B	2.6	0.7	5.0
Llama-3.2-3B	4.8	1.4	9.6
Llama-2-7B	7.5	4.5	37.9

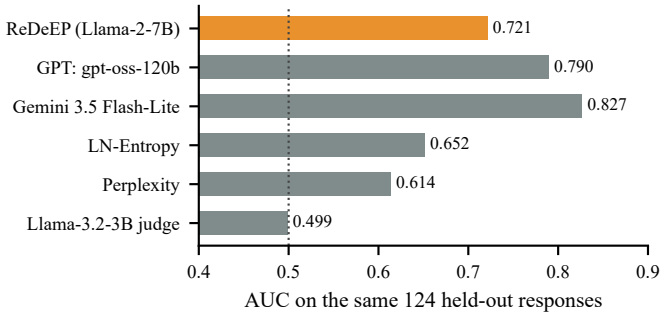


Fig. 9. All detectors on the same 124 held-out responses. The API judges are more accurate; a small local judge is at chance (dotted line, AUC 0.5).

TABLE VIII
JUDGES VS. REDEEP ON THE SAME 124 HELD-OUT RESPONSES

Detector	Kind	AUC	PCC	Rec.	F1
Gemini 3.5 Flash-Lite	Judge	**0.827**	0.671	0.914	0.865
GPT: gpt-oss-120b	Judge	0.790	0.465	0.929	0.778
ReDeEP (Llama-2-7B)	Internals	0.721	0.390	0.714	0.714
LN-Entropy	Uncertainty	0.652	–	–	–
Perplexity	Uncertainty	0.614	–	–	–
Llama-3.2-3B (local)	Judge	0.499	-0.005	0.071	0.127

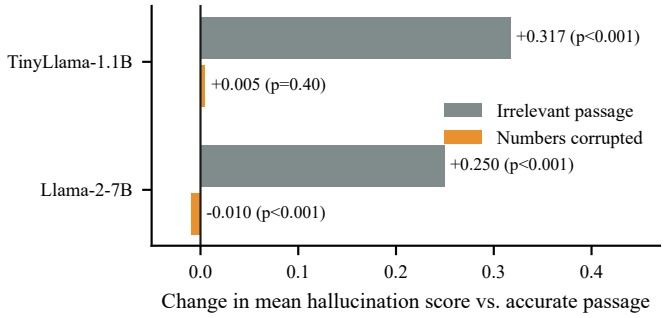


Fig. 10. Change in mean ReDeEP score relative to the accurate passage (paired Wilcoxon test, 150 responses per backbone).

for Llama-2-7B (both $p < 0.001$). The original labels then stop separating the responses (AUC ≈0.55; Fig. 10, Table IX). This is correct, because nothing in a response is supported by the new passage. The mechanism differs by size. In TinyLlama, ECS falls from 0.326 to 0.244, as copying heads find nothing useful to copy. In Llama-2-7B, ECS is flat and the increase comes from PKS (0.091 to 0.114): the larger model falls back on its own knowledge.

Corrupted numbers go unnoticed. For TinyLlama the score does not move (+0.005, $p = 0.40$). For Llama-2-7B it *falls* slightly (-0.010, $p < 0.001$) while ECS rises from 0.282 to 0.310, so the detector rates a contradicted response as more faithful. The corruption may not touch every response, so we checked the 122 responses that state a number the corrupted passage now contradicts. Their score rose in only 52% of cases for TinyLlama, which is chance, and 31% for Llama-2-7B. Detection AUC against the original labels is also unchanged under this condition, so the detector behaves as if the passage were still accurate.

C. Interpretation

ECS is a cosine similarity between hidden states. It measures whether the response is built from the same *content* the copying heads read, not whether individual facts agree. A changed number in an otherwise identical sentence plausibly moves the hidden state very little, so ReDeEP cannot see it. PKS does not compensate: the FFNs inject no more parametric knowledge when the passage's numbers change.

ReDeEP is therefore a detector of *topical* hallucination: a response that drifts away from what the context is about. It is not a detector of *factual* inconsistency with the context. In deployment, it should be paired with a fact-level check, such as matching numbers and entities between response and context, or a text-level judge. Notably, the paper's motivating example is itself a wrong number, which is the kind of error our experiment suggests ReDeEP is least sensitive to.

IX. DISCUSSION

What reproduces. The headline detection result reproduces closely on a laptop GPU, despite 4-bit weights and a sampled vocabulary for copying-head analysis. So do both mechanistic observations. The method also transfers to smaller and newer LLaMA models with little loss, which is encouraging for low-cost use.

What the pooled number hides. Per-task AUCs are 0.08–0.35 lower than pooled AUCs, and on data-to-text ReDeEP is unreliable for three of the four backbones. Evaluating hallucination detectors within task, or reporting both numbers, would give a more realistic picture of deployment performance.

Internals versus judges. Reading internals is cheap, local and needs no prompt, but on our data it is less accurate than asking a strong model. The two approaches fail differently. ReDeEP misses contradicted facts, while the judges are coarse and over-flag. A combination, for example ReDeEP as a free first-pass filter and a judge on flagged responses, is a natural next step.

X. LIMITATIONS AND THREATS TO VALIDITY

- *Quantisation:* the 3B and 7B models ran in 4-bit NF4, so their ECS and PKS approximate full precision. The close match with the paper suggests the effect is small.
- *Sampled copying-head scores:* OV circuits are scored on 500 sampled vocabulary tokens rather than the full vocabulary of 32k to 128k tokens.

TABLE IX
ROBUSTNESS: MEAN SCORE AND SIGNALS PER CONDITION

Backbone	Condition	Score	ECS	PKS	AUC
TinyLlama	Accurate	0.522	0.326	0.158	0.755
TinyLlama	Irrelevant	0.840	0.244	0.187	0.545
TinyLlama	Numbers corr.	0.528	0.330	0.160	0.747
Llama-2-7B	Accurate	0.521	0.282	0.091	0.710
Llama-2-7B	Irrelevant	0.770	0.288	0.114	0.564
Llama-2-7B	Numbers corr.	0.510	0.310	0.092	0.715

- *One generator:* all responses come from llama-2-7b-chat, so the other three backbones score text they did not write. This differs from the paper's matched setting.
- *Token level only:* we reproduce ReDeEP(token). The chunk-level variant, which the paper reports as stronger, is not evaluated.
- *Sample size:* 415 responses give fold standard deviations of 0.03–0.05 AUC, and differences below about 0.05 are within noise. Judge comparisons use 124 responses.
- *Robustness scope:* only numbers are corrupted. Entity swaps, negations and partially relevant passages are untested.
- *Judge access:* judges ran once, at default sampling settings, on free tiers. The GPT-family judge is the open-weights gpt-oss-120b, not a proprietary GPT-4-class model.

XI. Conclusion and Future Work

We reproduced ReDeEP on a 6 GB laptop GPU. The detector reaches an AUC of 0.739 on Llama-2-7B against 0.7325 in the paper, both of the paper's mechanistic findings replicate, and the method beats uncertainty baselines on four LLaMA models from 1.1B to 7B parameters. Its accuracy is insensitive to the number of selected heads and layers, and both of its signals contribute.

Our extension shows the method's boundary. ReDeEP catches responses that are off-topic for their context but not responses that contradict their context's numbers. Prompted judges from the Gemini and GPT families are more accurate, while small prompted judges do not work at all.

ReDeEP is therefore best used as a cheap, private first-line detector for topical hallucinations, paired with a fact-level check. Future work includes entity and negation corruptions, a numeric-consistency feature combined with ECS, the chunk-level variant, full-precision runs with backbone-matched responses on larger GPUs, an evaluation of AARF mitigation, and judge evaluation on all responses with repeated sampling.

References

[1] Z. Sun *et al.*, "ReDeEP: Detecting hallucination in retrieval-augmented generation via mechanistic interpretability," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025.

[2] C. Niu *et al.*, "RAGTruth: A hallucination corpus for developing trustworthy retrieval-augmented language models," in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024.

[3] H. Touvron *et al.*, "Llama 2: Open foundation and fine-tuned chat models," arXiv:2307.09288, 2023.

[4] A. Malinin and M. Gales, "Uncertainty estimation in autoregressive structured prediction," in *Proc. ICLR*, 2021.

[5] M. Geva, R. Schuster, J. Berant and O. Levy, "Transformer feed-forward layers are key-value memories," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[6] nostalgebraist, "Interpreting GPT: the logit lens," LessWrong, 2020.

Appendix A
Hyperparameters

Setting	Value
Top-k attended context tokens	10% of context length
Copying heads $\lvert\mathcal{A}\rvert$ / FFN layers $\lvert\mathcal{F}\rvert$	8 / 5
Vocabulary sample for OV analysis	500 tokens, seed 0
Maximum sequence length	1,024 tokens
Maximum prompt + response length	6,000 characters
Cross-validation	stratified 5-fold, seed 0
Single split	70/30, seed 0 (124 test responses)
Classifier	logistic regression
Feature scaling	standardised (z-score)
Threshold	best F1 on training fold
Quantisation (3B, 7B)	bitsandbytes NF4, fp16 compute
Robustness corruption factor	uniform in $[1.3, 3.0]$

Appendix B
Judge Prompt

System message given to every judge:

```
You are an expert fact-checker for retrieval-
augmented generation. You will be given the input
that was shown to an AI assistant (a task
instruction plus retrieved reference content) and
the assistant's response. Decide whether the
response contains any hallucination: a claim that
contradicts the reference content, or information
that is not supported by the reference content.
Judge only against the reference content, not
against your own world knowledge.
```

The user message contains the full RAG input and the response, and asks for a JSON reply with two fields: `hallucination_probability`, an integer from 0 to 100, and a one-sentence `reason`.

Appendix C
Reproducibility

All code, cached scores and logs are in the project repository. From the `redeep/` directory:

```
pip install -r requirements.txt
bash scripts/run_llama_family.sh   # detection
bash scripts/run_extensions.sh     # robustness
bash scripts/run_api_judges.sh     # GPT/Gemini
python paper/analysis.py           # ablations etc.
python paper/build_paper.py        # this report
```

Every number in this report is generated from the result files by `build_paper.py`; none is typed by hand.